Рекурсия. Выравнивание данных. Возврат структур и длинных типов из функций.

Рекурсия

Что такое рекурсия (в программировании)?

Рекурсия

Что такое рекурсия (в программировании)? - метод решения задачи, при котором функция, решающая нашу задачу вызывает сама себя с некоторым условием “остановки”.

Рекурсия

Что такое рекурсия (в программировании)? - метод решения задачи, при котором функция, решающая нашу задачу вызывает сама себя с некоторым условием “остановки”.

Примеры таких задач:

- Факториал: $a_n = n * a_{n - 1}$; $a_1 = 1$
- Числа Фибоначчи: $a_n = n + a_{n - 1}$; $a_1 = 1$

Рекурсия (факториал)

```c
long fact(long n) {
    long next_n = n  - 1;

    if (n <= 1) return 1;

   return n * fact(next_n);
}
```

```asm
fact:
        push    ebp
        mov     ebp, esp
        sub     esp, 4
        mov     eax, DWORD PTR [ebp+8]
        sub     eax, 1
        mov     DWORD PTR [ebp-4], eax
        cmp     DWORD PTR [ebp+8], 1
        jg      .L2
        mov     eax, 1
        jmp     .L3
.L2:
        push    DWORD PTR [ebp-4]
        call    fact
        add     esp, 4
        imul    eax, DWORD PTR [ebp+8]
.L3:
        leave
        ret
```

https://godbolt.org/z/f4aYEf3vj

Рекурсия (факториал)

Вспоминаем прошлый семинар.

Укажите, на каких строках находятся:

- Пролог, эпилог
- Передача аргументов в функцию, очистка стека

Для какого calling convention скомпилирован этот код?

```
1  fact:
2          push    ebp
3          mov     ebp, esp
4          sub     esp, 4
5          mov     eax, DWORD PTR [ebp+8]
6          sub     eax, 1
7          mov     DWORD PTR [ebp-4], eax
8          cmp     DWORD PTR [ebp+8], 1
9          jg      .L2
10         mov     eax, 1
11         jmp     .L3
12 .L2:
13         push    DWORD PTR [ebp-4]
14         call    fact
15         add     esp, 4
16         imul    eax, DWORD PTR [ebp+8]
17 .L3:
18         leave
19         ret
```

Рекурсия (факториал)

На каждый вызов вложенной функции мы выделяем фрейм под локальные переменные.

Это ограничивает глубину рекурсии. Как мы можем этого избежать?

```
1  fact:
2          push    ebp
3          mov     ebp, esp
4          sub     esp, 4
5          mov     eax, DWORD PTR [ebp+8]
6          sub     eax, 1
7          mov     DWORD PTR [ebp-4], eax
8          cmp     DWORD PTR [ebp+8], 1
9          jg      .L2
10         mov     eax, 1
11         jmp     .L3
12 .L2:
13         push    DWORD PTR [ebp-4]
14         call    fact
15         add     esp, 4
16         imul    eax, DWORD PTR [ebp+8]
17 .L3:
18         leave
19         ret
```

Хвостовая рекурсия

Это частный случай рекурсивной функции, в которой вызов функции является последней операцией.

Пример:

```
void print_all (long n) {
    if (n <= 1)
        return;

    printf ("%d\n", n);

    return print_all (n - 1);
}
```

Хвостовая рекурсия

Является ли наш факториал хвостовой рекурсией?

```
long fact(long n) {
    long next_n = n  - 1;

    if (n <= 1) return 1;

   return n * fact(next_n);
}
```

Хвостовая рекурсия

Является ли наш факториал хвостовой рекурсией?

Nope. После вызова fact (n - 1), мы должны произвести умножение на n перед возвратом.

```
long fact(long n) {
    long next_n = n  - 1;

    if (n <= 1) return 1;

   return n * fact(next_n);
}
```

Хвостовая рекурсия

Является ли наш факториал хвостовой рекурсией?

Nope. После вызова fact (n - 1), мы должны произвести умножение на n перед возвратом.

Легко это увидеть, посмотреть на ассемблер.

Как перепишем?

```
1  fact:
2          push    ebp
3          mov     ebp, esp
4          sub     esp, 4
5          mov     eax, DWORD PTR [ebp+8]
6          sub     eax, 1
7          mov     DWORD PTR [ebp-4], eax
8          cmp     DWORD PTR [ebp+8], 1
9          jg      .L2
10         mov     eax, 1
11         jmp     .L3
12 .L2:
13         push    DWORD PTR [ebp-4]
14         call    fact
15         add     esp, 4
16         imul    eax, DWORD PTR [ebp+8]
17 .L3:
18         leave
19         ret
```

Хвостовая рекурсия

Используем аргумент-аккумулятор.

```c
long fact_tail(long n, long acc) {
    long next_n = n  - 1;
    long next_acc = acc * n;

    if (n <= 1)
        return acc;

   return fact_tail(next_n, next_acc);
}
```

```asm
fact_tail:
        push    ebp
        mov     ebp, esp
        sub     esp, 8
        mov     eax, DWORD PTR [ebp+8]
        mov     edx, DWORD PTR [ebp+12]
        lea     ecx, [eax-1]
        mov     DWORD PTR [ebp-4], ecx
        mov     ecx, edx
        imul    ecx, eax
        mov     DWORD PTR [ebp-8], ecx
        cmp     eax, 1
        jg      .L5
        mov     eax, edx
        jmp     .L7
.L5:
        mov     eax, DWORD PTR [ebp-8]
        mov     DWORD PTR [ebp+12], eax
        mov     eax, DWORD PTR [ebp-4]
        mov     DWORD PTR [ebp+8], eax
        leave
        jmp     fact_tail
.L7:
        leave
        ret
```

Выравнивание данных

Выравнивание – это требование, чтобы адрес, по которому располагаются данные, был кратен определенному значению.

	IA-32		x86_64	
	Linux	Windows	Linux	Windows
`char`	-	-	-	-
`short`	2	2	2	2
`int`	4	4	4	4
`long`	4	4	8	4
`ptr`	4	4	8	8
`float`	4	4	4	4
`double`	4	8	8	8
`long double`	4	4	8	8

Выравнивание данных (структуры)

Рассмотрим следующую структуру.

Попробуйте предположить, какие у нее размер и выравнивание.

```
struct S {
    char c;
    int a;
};
```

Выравнивание данных (структуры)

Рассмотрим следующую структуру.

Попробуйте предположить, какие у нее размер и выравнивание.

```c
struct S {
    char c;
    int a;
};

int main () {
    printf ("%d\n", sizeof(struct S));
    printf ("%d\n", alignof(struct S));
}
```

```
Program returned: 0
Program stdout
8
4
```

https://godbolt.org/z/MszTjv811

Выравнивание данных (структуры)

В данном случае поле “a” должно быть выровнено на 4 байта => компилятор вставит “padding” в 3 байта после “c”.

Поэтому, размер структуры равен 8 байтам.

```
struct s {
    char c;
    int a;
};
```

Правила выравнивания агрегатных типов данных

- Требование к выравниванию структуры определяется как максимальное из требований к выравниванию полей.

- Требование к выравниванию объединения определяется как максимальное из требований выравниванию полей

- Требование к выравниванию массива совпадает с требованием к выравниванию его элементов

- Если требование к выравниванию структуры/объединения равно K, то размер должен быть кратен K

Выравнивание данных (стек)

Некоторые инструкции x86 требуют определенного выравнивания для стека.

Поэтому, например в System-V ABI существует требование на выравнивание стека в 16 байт перед вызовом функций.

Поэкспериментируем с выравниванием стека на примере нашего факториала:

https://godbolt.org/z/M7nqW7jsq (без специальных флагов)
https://godbolt.org/z/oscP17aPh (со специальными флагами)

Возврат структур

Предположим, мы решили написать следующий код:

https://godbolt.org/z/64YWGqzar

```
struct student {
    int id;
    int group;
};

struct student make_student (int id, int group) {
    struct student result = {
        .id = id,
        .group = group
    };

    return result;
}

int main () {
    struct student new_student = make_student (1, 2510);

    return 0;
}
```

Возврат структур

Посмотрим подробнее на эту функцию:

- Распишем структуру стека: где расположены входные параметры, локальные переменные, возвращаемое значение

```
make_student:
        push    ebp
        mov     ebp, esp
        sub     esp, 8
        mov     eax, DWORD PTR [ebp+12]
        mov     DWORD PTR [ebp-8], eax
        mov     eax, DWORD PTR [ebp+16]
        mov     DWORD PTR [ebp-4], eax
        mov     ecx, DWORD PTR [ebp+8]
        mov     eax, DWORD PTR [ebp-8]
        mov     edx, DWORD PTR [ebp-4]
        mov     DWORD PTR [ecx], eax
        mov     DWORD PTR [ecx+4], edx
        mov     eax, DWORD PTR [ebp+8]
        leave
        ret     4
```

Возврат структур

Посмотрим подробнее на эту функцию:

```c
struct student make_student (int id, int group) {
    struct student result = {
        .id = id,
        .group = group
    };

    return result;
}
```

```
make_student:
        push    ebp
        mov     ebp, esp
        sub     esp, 8
        mov     eax, DWORD PTR [ebp+12]
        mov     DWORD PTR [ebp-8], eax
        mov     eax, DWORD PTR [ebp+16]
        mov     DWORD PTR [ebp-4], eax
        mov     ecx, DWORD PTR [ebp+8]
        mov     eax, DWORD PTR [ebp-8]
        mov     edx, DWORD PTR [ebp-4]
        mov     DWORD PTR [ecx], eax
        mov     DWORD PTR [ecx+4], edx
        mov     eax, DWORD PTR [ebp+8]
        leave
        ret     4
```

Возврат структур

Посмотрим подробнее на эту функцию:

```c
struct student make_student (int id, int group) {
    struct student result = {
        .id = id,
        .group = group
    };

    return result;
}
```

```asm
make_student:
        push    ebp
        mov     ebp, esp
        sub     esp, 8
        mov     eax, DWORD PTR [ebp+12]
        mov     DWORD PTR [ebp-8], eax
        mov     eax, DWORD PTR [ebp+16]
        mov     DWORD PTR [ebp-4], eax
        mov     ecx, DWORD PTR [ebp+8]
        mov     eax, DWORD PTR [ebp-8]
        mov     edx, DWORD PTR [ebp-4]
        mov     DWORD PTR [ecx], eax
        mov     DWORD PTR [ecx+4], edx
        mov     eax, DWORD PTR [ebp+8]
        leave
        ret     4
```

Возврат структур

Это что?

```c
struct student make_student (int id, int group) {
    struct student result = {
        .id = id,
        .group = group
    };

    return result;
}
```

```asm
make_student:
        push    ebp
        mov     ebp, esp
        sub     esp, 8
        mov     eax, DWORD PTR [ebp+12]
        mov     DWORD PTR [ebp-8], eax
        mov     eax, DWORD PTR [ebp+16]
        mov     DWORD PTR [ebp-4], eax
        mov     ecx, DWORD PTR [ebp+8]
        mov     eax, DWORD PTR [ebp-8]
        mov     edx, DWORD PTR [ebp-4]
        mov     DWORD PTR [ecx], eax
        mov     DWORD PTR [ecx+4], edx
        mov     eax, DWORD PTR [ebp+8]
        leave
        ret     4
```

Возврат структур

Это скрытый первый аргумент, который используется для возврата структур из функций.

По указателю, который был передан через этот скрытый аргумент, мы кладем поля нашей результирующей структуры.

```
1   make_student:
2           push    ebp
3           mov     ebp, esp
4           sub     esp, 8
5           mov     eax, DWORD PTR [ebp+12]
6           mov     DWORD PTR [ebp-8], eax
7           mov     eax, DWORD PTR [ebp+16]
8           mov     DWORD PTR [ebp-4], eax
9           mov     ecx, DWORD PTR [ebp+8]
10  id      mov     eax, DWORD PTR [ebp-8]
11  group   mov     edx, DWORD PTR [ebp-4]
12  id      mov     DWORD PTR [ecx], eax
13  group   mov     DWORD PTR [ecx+4], edx
14          mov     eax, DWORD PTR [ebp+8]
15          leave
16          ret     4
```

Возврат структур

Откуда берется скрытый аргумент?

Посмотрим на вызов нашей функции:

В качестве скрытого аргумента передается адрес для памяти, специально выделенной под эту структуру.

```
main:
        push    ebp
        mov     ebp, esp
        sub     esp, 8
        lea     eax, [ebp-8]
        push    2510
        push    1
        push    eax
        call    make_student
        add     esp, 8
        mov     eax, 0
        leave
        ret
```

Возврат структур (итог)

Для того, чтобы вернуть из функции структуру, нам нужно сделать следующие шаги.

1) Caller выделяет на стеке память под возвращаемую структуру
2) Передает указатель на эту память callee
3) Код функции сгенерирован так, чтобы положить результат по этому адресу
4) Адрес возвращаемой структуры помещается в eax